

once pinned down, the system—which is the set of type declarations, the compiler, and the runtime system -- makes it as hard as it can for you to change those decisions, on the assumption that all such changes are mistakes you're inadvertently making. The effect is like having governors (speed limiters) fitted in your off-roader to prevent fools (you?) from doing too much damage. Hackers don't like a language that talks down to them. Hackers just want power.

C/C++. If you get into serious programming, you will have to learn C, the core language of UNIX. C++ is very closely related to C; if you know one, learning the other will not be difficult. Neither language is a good one to try learning as your first, however.

Perl. Another language of particular importance to hackers is Perl, which is worth learning for practical reasons. It is very widely used for active Web pages and systems administration, so that even if you never write Perl, you should learn to read it.

Lisp. The truly serious hacker should consider learning Lisp. Lisp is worth learning for the profound enlightenment experience you will have when you finally get it; that experience will make you a better programmer for the rest of your days, even if you never actually use Lisp a lot.

Get Linux

The single most important step any newbie can take toward acquiring hacker skills is to get a copy of Linux or one of the BSD-Unixes, install it on a personal machine, and run it. Yes, there are other operating systems in the world besides UNIX/Linux. But they're distributed in binary—you can't read the code and you can't modify it. Trying to learn to hack on a Windows machine is like trying to learn to dance while wearing a body cast. Besides, UNIX is the operating system of the Internet. While you can learn to use the Internet without knowing UNIX, you can't be an Internet hacker without understanding UNIX. For this reason, the hacker culture today is pretty strongly UNIX-centred.

So, bring up a Linux. Learn it. Run it. Tinker with it. Talk to the Internet with it. Read the code and modify it. You'll get better programming tools (including C, Lisp, Python and Perl) than any Microsoft operating system can dream of. You'll have fun and you'll soak up more knowledge than you realise, until you look back on it as an ace hacker.

So are you a hacker?

You must earn the title of 'hacker', rather than just claim it. The same essay by Eric S Raymond that I mentioned earlier offers some invaluable tips on how to earn your status in hackerdom. Among others, there are two main things you can do to be respected by hackers (for a complete list, read 'How to become a hacker' by Eric S Raymond at <http://catb.org/~esr/faqs/hacker-howto.html>).

[org/~esr/faqs/hacker-howto.html](http://catb.org/~esr/faqs/hacker-howto.html)).

Write open source software. The first (the most central and most traditional) is to write programs that other hackers think are fun or useful, and give the program sources to the whole hacker culture to use. Hackerdom's most revered demigods are people who have written large, capable programs that met a widespread need, and given them away, so that now everyone uses them.

Help test and debug open source software. They also serve hackerdom who stand and debug open source software. In this imperfect world, we will inevitably spend most of our software development time in the debugging phase. That's why any open source author will tell you that good beta-testers (who know how to describe symptoms clearly, localise problems well, can tolerate bugs in a quickie release, and are willing to apply a few simple diagnostic routines) are worth their weight in rubies. Even one of these can make the difference between a debugging phase that's a protracted, exhausting nightmare and one that's merely a salutary nuisance.

If you're a newbie, try to find a program under development that you're interested in and be a good beta-tester. There's a natural progression from helping test programs to helping debug them and then on to helping modify them. You'll learn a lot this way and generate good karma with people who will help you later on.

To end, I will have to quote Eric S Raymond yet again, since he puts it so beautifully: "We half-joke about 'world domination', but the only way we will get there is by serving the world. That means you and I; and that means learning how to think about what we do in a fundamentally new way, and ruthlessly reducing the user-visible complexity of the default environment to an absolute minimum.

"Computers are tools for human beings. Ultimately, therefore, the challenges of designing hardware and software must come back to designing for human beings—all human beings. This path will be long, and it won't be easy. But we owe it to ourselves and each other to do it right. May the open source be with you!" 

References

- [1] *How to Become a Hacker*, an essay by Eric Steven Raymond
- [2] *Hackers: Heroes of the Computer Revolution*, by Steven Levy
- [3] *The Daemon, the GNU, and the Penguin*, by Peter H. Salus
- [4] *OpenSources*, by Chris DiBona, Sam Ockman and Mark Stone
- [5] *The New Hacker's Dictionary*, by Eric S Raymond, MIT Press
- [6] *Biopunk: DIY Scientists Hack the Software of Life*, by Marcus Wohlsen

By: Vivek Shangari

The author is a hard-core hacker, if there was one. He always thinks programs and holds a piece of code in his head all the time. You can reach him at vivek@codecraft.in.